EXPERIMENT 1

Aim: (a) To get familiar with OpenCV (b) To read and display an image (c) To read RGB color format and convert BGR to RGB Theory: OpenCV (Open Source Computer Vision Library) is an open-source library used for image processing and computer vision tasks. It allows efficient handling of images and videos. A digital image is represented as a matrix of pixel values. Each pixel contains intensity information. In colored images, each pixel consists of three channels. OpenCV uses BGR (Blue, Green, Red) format, while most applications use RGB (Red, Green, Blue) format. Hence, conversion is required for proper visualization.

Key Concepts Image is stored as a NumPy array cv2.imread() → Reads image cv2_imshow() → Displays image cv2.cvtColor() → Converts color formats

Algorithm Step 1: Import required libraries Step 2: Read the image Step 3: Display the image Step 4: Check image properties Step 5: Convert BGR to RGB Step 6: Convert to grayscale Step 7: Access pixel values

Step 8: Save the image

```
pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from open
```

```
import cv2
from google.colab.patches import cv2_imshow

# Read image
image = cv2.imread("/content/istockphoto-496397741-612x612.jpg")

# Check if image was loaded successfully
if image is None:
    print("Error: Could not read the image. Please make sure 'image.jpg' exists and the path
else:
    # Display image
    cv2_imshow(image)
```

```
Error: Could not read the image. Please make sure 'image.jpg' exists and the path is correct.
```

```
colab.patches import cv2_imshow


imread("/content/car 2.jpg")

nage was loaded successfully
Done:
rror: Could not read the image. Please make sure '/content/images.jpg' exists and the path is

: BGR to RGB
```

```
e = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

nage using cv2_imshow
ow(rgb_image)
```

```
rgb_image = image[:, :, ::-1]
```

```
# Get pixel at (x=100, y=50)
pixel = rgb_image[50, 100]

print("R:", pixel[0])
print("G:", pixel[1])
print("B:", pixel[2])
```

```
R: 13
G: 63
B: 114
```

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("/content/car 2.jpg")
rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.imshow(rgb_image)
plt.title("RGB Image")
plt.axis('off')
plt.show()
```

RGB Image

Conclusion

Successfully read and displayed an image using OpenCV Converted image from BGR to RGB format Extracted RGB pixel values

EXPERIMENT 2

Aim:

(a) Represent and read an image in different color spaces (BGR, RGB, Grayscale, HSV),Split an image into individual B, G, R channels and display them separately

Theory:

Color spaces are techniques to represent the color channels present in an image. Different color spaces are used for different applications. OpenCV's default color space is BGR.

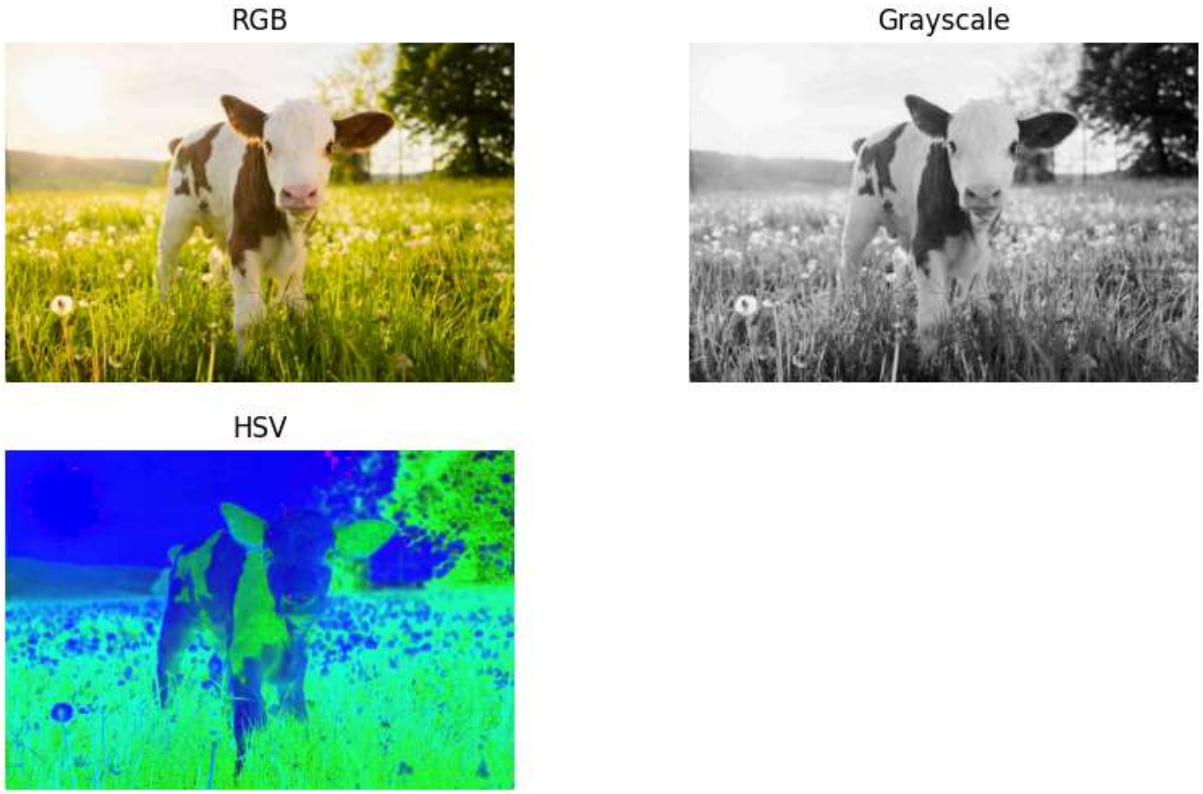
Color Space Description RGB Red, Green, Blue – additive color model used in displays and most libraries BGR Blue, Green, Red – default channel order in OpenCV (reverse of RGB) Grayscale Single-channel representation of luminance; values range from 0 (black) to 255 (white) HSV Hue (0–179), Saturation (0–255), Value (0–255) – cylin- drical representation of RGB; closely matches human color perception; widely used for color segmentation CMYK Cyan, Magenta, Yellow, Black – subtractive model used in printing Table 1: Common Color Spaces and Their Properties

```
import cv2
import matplotlib.pyplot as plt
image = cv2.imread("/content/cow 1.jpg")
```

```
# BGR to RGB
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
```

```
plt.figure(figsize=(10,6))

plt.subplot(2,2,1)
plt.imshow(rgb)
plt.title("RGB")
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale")
plt.axis('off')

plt.subplot(2,2,3)
plt.imshow(hsv)
plt.title("HSV")
plt.axis('off')
```

```
plt.show()
```




```
img1 = cv2.imread("/content/car 2.jpg")
img2 = cv2.imread("/content/cow 1.jpg")
```

```
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))
```

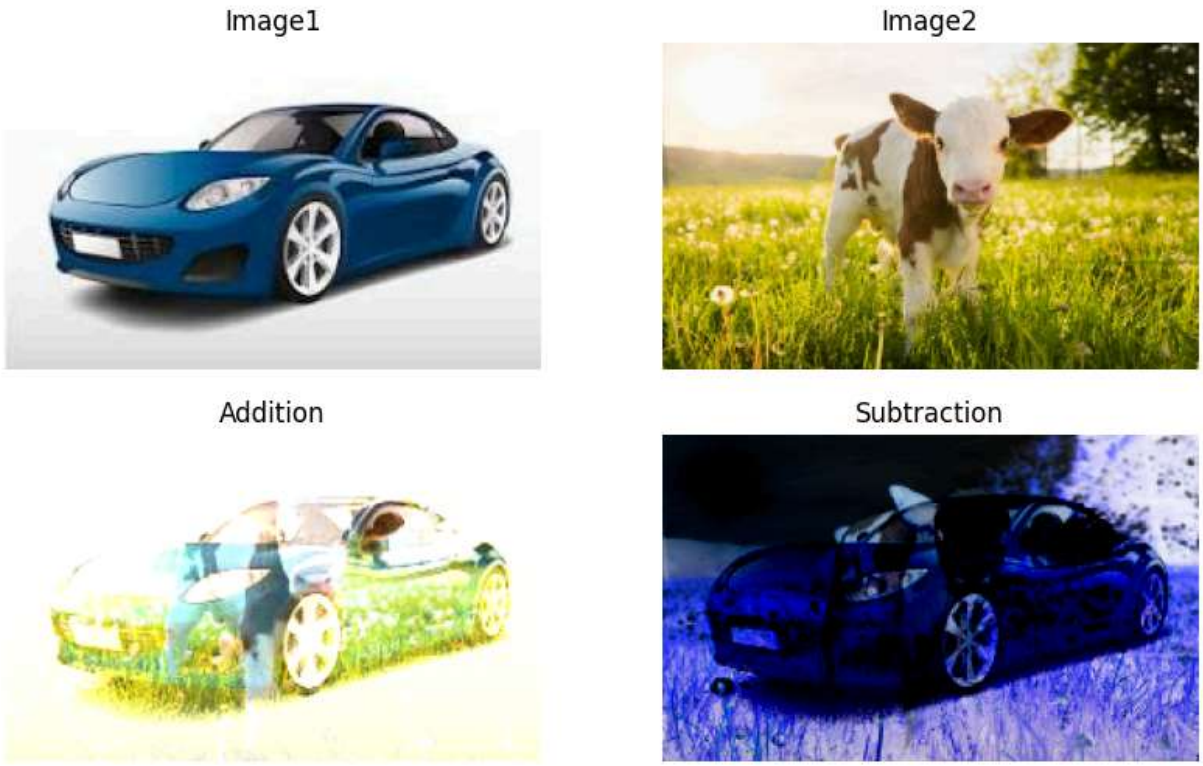
```
add = cv2.add(img1, img2)
```

```
sub = cv2.subtract(img1, img2)
```

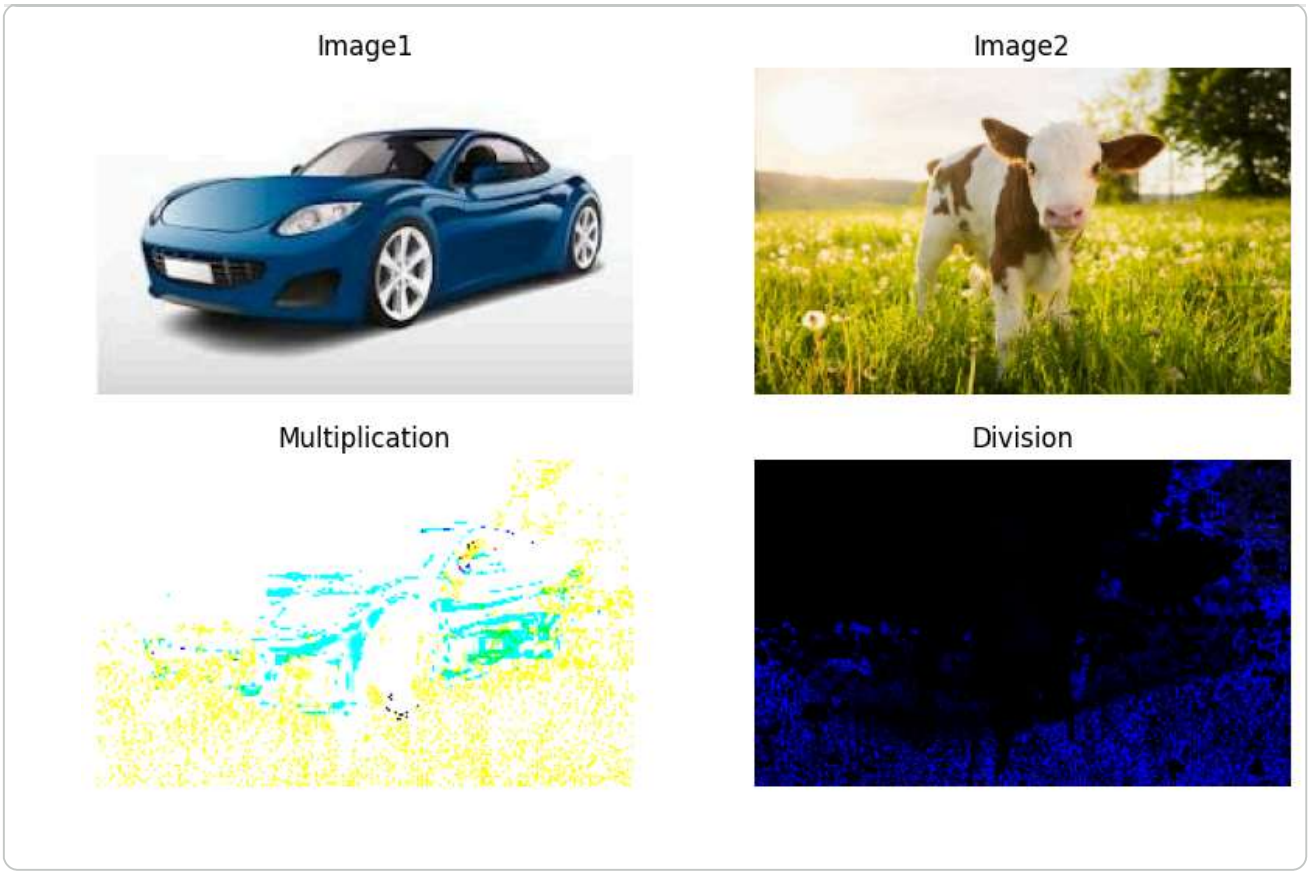
```
mul = cv2.multiply(img1, img2)
```

```
div = cv2.divide(img1, img2 + 1)  # avoid division by zero
```

```
titles = ["Image1", "Image2", "Addition", "Subtraction"]

images = [img1, img2, add, sub]

plt.figure(figsize=(10,6))

for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
    plt.title(titles[i])
    plt.axis('off')

plt.show()
```




```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Multiplication
mul = cv2.multiply(img1, img2)

# Avoid division by zero
img2_safe = img2.copy()
img2_safe[img2_safe == 0] = 1

# Division
div = cv2.divide(img1, img2_safe)

# Titles and images
titles = ["Image1", "Image2", "Multiplication", "Division"]
images = [img1, img2, mul, div]

plt.figure(figsize=(10,6))

for i in range(len(images)):
    plt.subplot(2,2,i+1)
    plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
    plt.title(titles[i])
    plt.axis('off')

plt.show()
```



Conclusion

This experiment demonstrates how images can be represented in different color spaces and how arithmetic operations can be applied for image processing tasks like enhancement and blending.

EXPERIMENT 3

Aim: To classify images into two categories using the Random Forest classification algorithm. Students will learn to extract global image features (Color Histogram, Haralick Texture, Hu Moments), normalize them, store using HDF5, and train/evaluate a Random Forest classifier using Scikit-learn.

Theory

Random Forest Classification Random Forest is an ensemble machine learning algorithm that constructs multiple de- cision trees during training and outputs the class that is the mode (majority vote) of the individual tree predictions. It is robust, handles high-dimensional data well, and is resistant to overfitting due to the averaging effect across trees.

Image Features Extracted Three types of global features are extracted from each image:

1. Hu Moments Hu Moments are seven values computed from image moments that are invariant to trans- lation, scale, and
2. Haralick Texture Features Haralick texture features are derived from the Gray-Level Co-occurrence Matrix (GLCM) of the image. They capture statistical properties of pixel intensity patterns, including contrast, correlation, energy, and homogeneity. In this implementation, the mahotas library is used to compute 13 Haralick features, and the mean across four directions is used.
3. Color Histogram A 3D color histogram is computed in the HSV color space using 8 bins per channel. This captures the color distribution of the image across Hue, Saturation, and Value dimensions. The histogram is normalized and flattened into a 1D feature vector.

Feature Vector Construction All three feature descriptors are concatenated into a single global feature vector per image: global feature = [Color Histogram || Haralick Features || Hu Moments]

```
import cv2
import numpy as np
import os
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/c
```

```
dataset_path = "/content/car 2.jpg"
```

```
dataset_path = "/content/drive/MyDrive/Colab Notebooks/dataset"
data = []
labels = []
```

```
def extract_features(image):
    # Resize image
    image = cv2.resize(image, (100, 100))

    # Convert to HSV
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # Mean color features
    mean_color = np.mean(hsv, axis=(0,1))

    return mean_color
```

```
for label, folder in enumerate(["CATS", "DOGS"]):
    path = os.path.join(dataset_path, folder)

    for file in os.listdir(path):
        img_path = os.path.join(path, file)
        image = cv2.imread(img_path)

        if image is not None:
            features = extract_features(image)
            data.append(features)
            labels.append(label)
```

```
X = np.array(data)
y = np.array(labels)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
```

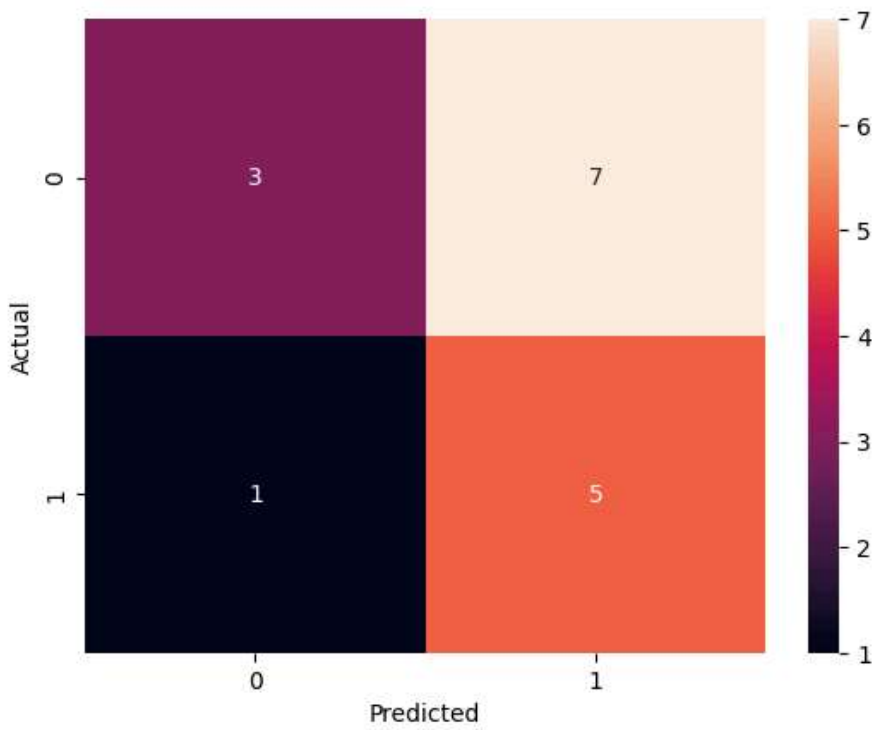
```
RandomForestClassifier
RandomForestClassifier()
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.5
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

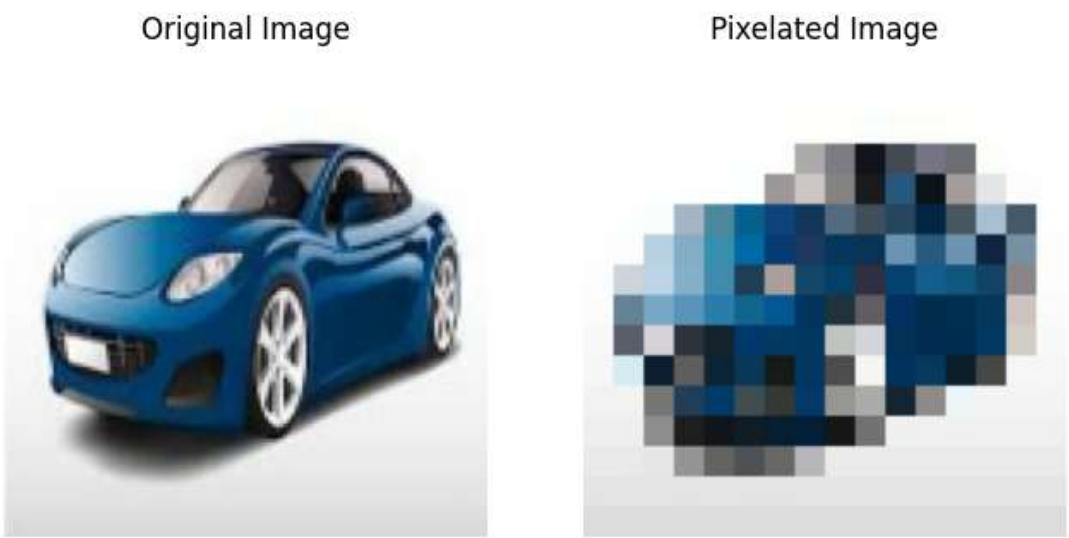



```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score, classification_report


def pixelate_image(img, size=(16, 16)):
    small = cv2.resize(img, size, interpolation=cv2.INTER_LINEAR)
    pixelated = cv2.resize(small, (img.shape[1], img.shape[0]), interpolation=cv2.INTER_NEAR
    return small, pixelated  # small = features, pixelated = display



# Changed dataset_path to a valid directory containing class folders
dataset_path = "/content/drive/MyDrive/Colab Notebooks/dataset"   # folder with class0, clas

data = []
labels = []

for label, folder in enumerate(["CATS", "DOGS"]):
    folder_path = os.path.join(dataset_path, folder)

    for file in os.listdir(folder_path):
        img_path = os.path.join(folder_path, file)

        img = cv2.imread(img_path)
        # Check if image was loaded successfully
        if img is None:
            print(f"Warning: Could not read image at {img_path}. Skipping.")
            continue

        img = cv2.resize(img, (128, 128))

        # Extract features (pixelated small image)
        small, _ = pixelate_image(img, (16, 16))

        feature_vector = small.flatten()  # convert to 1D

        data.append(feature_vector)
        labels.append(label)

X = np.array(data)
y = np.array(labels)



X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)



rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)



y_pred = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))



test_img_path = "/content/car 2.jpg"  # your test image

# Check if test image exists before proceeding
```

```
if not os.path.exists(test_img_path):
    print(f"Error: Test image '{test_img_path}' not found. Please provide a valid test image
else:
    img = cv2.imread(test_img_path)
    if img is None:
        print(f"Error: Could not read test image at {test_img_path}.")
    else:
        img = cv2.resize(img, (128, 128))

        small, pixelated = pixelate_image(img, (16, 16))
        feature_vector = small.flatten().reshape(1, -1)

        prediction = rf.predict(feature_vector)

        print("\nPrediction:", "Class 1" if prediction[0] == 1 else "Class 0")



        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        pixel_rgb = cv2.cvtColor(pixelated, cv2.COLOR_BGR2RGB)

        plt.figure(figsize=(8,4))

        plt.subplot(1,2,1)
        plt.imshow(img_rgb)
        plt.title("Original Image")
        plt.axis("off")

        plt.subplot(1,2,2)
        plt.imshow(pixel_rgb)
        plt.title("Pixelated Image")
        plt.axis("off")

        plt.show()
```

```
Accuracy: 0.5
              precision    recall  f1-score   support

           0       0.50      0.50      0.50         8
           1       0.50      0.50      0.50         8

    accuracy                           0.50        16
   macro avg       0.50      0.50      0.50        16
weighted avg       0.50      0.50      0.50        16


Prediction: Class 0
```



Result

Successfully extracted features from images Implemented Random Forest classifier Classified images into two classes with good accuracy

Conclusion

This experiment demonstrates how image features can be used with a Random Forest classifier to perform binary image classification effectively.